

NowBot

Real-Time Info Chatbot Challenge

Career Fair • 2-Day Challenge

Duration	2 Days
Difficulty	Beginner → Intermediate
AI Allowed	Yes — use any LLM API, framework, or AI tool you like
Deliverable	A chatbot that answers live sports and weather questions using real APIs

Overview

Most chatbots only know what they were trained on. NowBot is different, it knows what is happening right now.

Your task is to build a conversational assistant that pulls live data from two real-world APIs and uses an LLM to turn that raw data into clear, natural answers.

A user should be able to ask anything from 'What is the weather like in Muscat?' What happened in the NBA last night? and get an accurate, up-to-date response.

Sample Questions NowBot Should Handle

- What is the weather like in New York right now?
- Should I bring an umbrella to Muscat today?
- What were the latest NBA scores?
- Any big sports news in the NFL this week?
- What is the forecast for the next 3 days in Tokyo?
- Who won the match last night?

Your Data Sources

You are required to use both of the following APIs. No third-party wrappers — call them directly.

Sports News API - NewsAPI.org

URL: `https://newsapi.org/s/us-sports-news-api`

Purpose: Fetch the latest US sports news articles, headlines, and match updates

Notes: **Free** tier available. Requires an API key

Weather API - National Weather Service

URL: `https://api.weather.gov`

Purpose: Fetch real-time weather and multi-day forecasts based on geographic coordinates

Notes: **Completely** free; no API key required.

Challenge Levels

Each level builds on the previous one. You must complete a level before moving to the next level.

LEVEL 1 - Live Data, Real Answers

Requirements

- Connect to both APIs and successfully fetch live data
- Accept a user question via CLI or notebook
- Detect whether the question is about weather or sports
- Fetch the relevant data and pass it to an LLM to generate a natural answer
- Display the answer to the user clearly

Technical Hints

- *For weather: use /points/{lat},{lon} first, then follow the forecastUrl from the response*
- *For sports: fetch top headlines from the sports category and pass them as context*
- *Use simple keyword detection to route the question (e.g. 'weather', 'rain', 'forecast' vs 'score', 'game', 'match')*
- *Hardcode a city or location for Level 1 no need for dynamic location yet*
- *Tell the LLM: Answer only based on the data provided. Do not use your training knowledge.*

LEVEL 2 - Smarter Routing

Requirements

Technical Hints

• Detect the user's intended location from their question dynamically • Convert city names to coordinates to use with the weather API • Filter sports news by sport type when the user specifies (e.g. NBA, NFL, soccer) • Handle ambiguous questions gracefully ask a clarifying question if needed • Maintain conversation context so follow-up questions work correctly	• <i>Use a geocoding API (e.g. nominatim.openstreetmap.org — free, no key) to convert city names to lat/lon</i> • <i>For sports filtering: search article titles/descriptions for sport keywords before sending to the LLM</i> • <i>Keep a messages array and append each turn to maintain context across calls</i> • <i>If the user asks, 'what about tomorrow?' the bot should remember the last city they mentioned</i> • <i>Test with tricky inputs: misspelled cities, vague questions, mixed weather + sports in one message</i>
--	--

LEVEL 3 - A Real Product	
Requirements	Technical Hints

- | | |
|--|--|
| <ul style="list-style-type: none">• Build a full chat UI — web, mobile, or desktop• Show a loading state while fetching live data• Display the source of the answer (which API, which article, which forecast period)• Handle API failures gracefully — inform the user instead of crashing• Bonus: let the user set a default location that persists across the session | <ul style="list-style-type: none">• <i>A simple HTML + JS page, Streamlit app, or React frontend all work well</i>• <i>Show the raw API data fetched alongside the LLM answer for transparency</i>• <i>Wrap all API calls in try/catch and show a friendly error message on failure</i>• <i>For the bonus: store the preferred location in localStorage or a session variable</i>• <i>Consider showing a weather icon or sport emoji to make the UI feel alive</i> |
|--|--|

Judging Criteria

Maximum score: 100 points. Judges will ask live questions during the demo.

Criteria	What We Look For	Points
API Integration	Both APIs are called correctly and return live data	20 pts
Answer Accuracy	Responses reflect real current data, not hallucinated info	20 pts
Smart Routing	Bot correctly identifies weather vs sports and handles edge cases	20 pts
Conversation Quality	Follow up questions work, context is maintained across turns	15 pts
Code Quality	Clean, readable, well structured code, NO HARD CODED API KEYS	15 pts
UI / UX (Level 3)	Interface is clean, shows sources, and loads indicators	10 pts

- Extra **20** points would be given to students who are able to containerize the app and provide a good high level readme file on how to setup and start. If you decide to host and deploy, that would be an extra **10** points.

- There is no need to provide an API key.

Tips for Success

- Read both API docs before writing a single line of code for the weather API has a two-step flow.
- Test your API calls in a REST client (Postman, Insomnia, or curl) first.
- Let the LLM summarize and explain the data using your code to fetch and filter it.
- A bot that says 'I could not get the latest data' is better than the one that invents scores.
- The NewsAPI free tier has rate limits.